

Education DRM Service

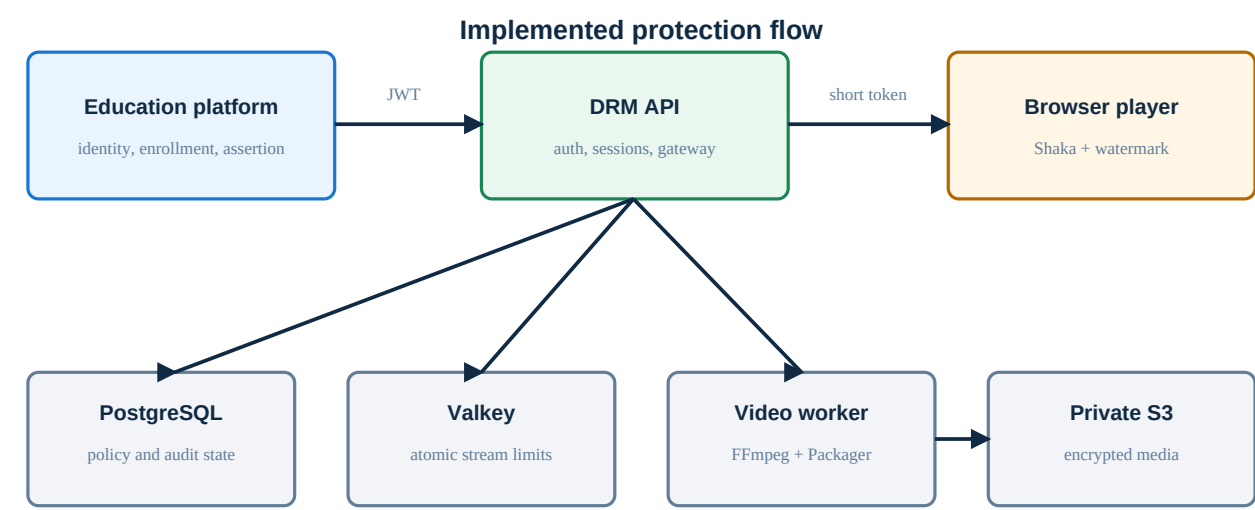
Architecture, integration, verified controls, and honest security limits

IMPLEMENTATION STATUS	Hardened development build	DATE	27 Sep 2026
-----------------------	----------------------------	------	-------------

Executive Summary

This repository is the protected-video subsystem for an external education platform. It handles scoped video ingest, encrypted DASH packaging, playback sessions, private delivery, a development license provider, watermark evidence, webhooks, and audit data. The education platform remains responsible for users, courses, enrollment, payments, and the decision that a student may watch.

Important: this system reduces theft and account sharing; it cannot guarantee that video cannot be recorded. No browser can stop an external camera, and ClearKey is not production DRM.



Responsibility Boundary

Education platform	DRM service
Login, identity, enrollment, course access, payments, assertion signing	Upload scope, processing, keys, sessions, device/stream policy, media gateway, licenses, watermark evidence

Protection Flow

1. Upload and processing

The backend registers an asset. The API generates a tenant-prefixed UUID key and a short-lived presigned PUT URL. The caller uploads directly, then explicitly completes the upload. The API checks object metadata before it atomically queues processing.

The worker streams the source into an isolated temporary directory, validates it with ffprobe, enforces size and duration bounds, transcodes without upscaling, packages encrypted DASH/CENC output, uploads to private S3-compatible storage, and clears decrypted key buffers.

2. Playback authorization

The education backend signs a one-use JWT bound to application, user, asset, and optional device. Production requires this assertion. The service verifies JWKS signature and claims, consumes the jti atomically, serializes device admission in PostgreSQL, and reserves concurrent playback atomically in Valkey.

3. Protected delivery

The browser receives a short-lived bearer token, never storage credentials. Each manifest, segment, range, and license request is checked against the persisted active session. Token renewal rotates the token ID, immediately invalidating the previous token. Revocation and expiry therefore affect delivery, not only the player UI.

Implemented Security Controls

- Tenant-scoped database operations and application ownership validation
- Argon2 application credentials plus separate constant-time admin authentication
- AES-256-GCM encryption for content keys and webhook secrets, with key versions
- Strict upload completion, media metadata checks, streaming storage I/O, and byte-range validation
- Signed HTTPS-only webhooks with encrypted per-endpoint secrets, redirect blocking, and private-network rejection
- HTTPS enforcement, strict CORS, CSP/security headers, input/body limits, and rate limiting
- Read-only non-root application containers, internal data network, dropped capabilities, locked dependencies, and CI checks

DRM and Watermark Status

Control	Current status	Meaning
ClearKey	Development only	Protocol and integration testing; raw keys reach browser JavaScript/CDM boundary.
Widevine	External integration pending	Requires approved provider, credentials, packaging metadata, and device conformance tests.
Browser watermark	Implemented	Signed rotating overlay with masked identity and persisted trace code.
Burned-in watermark	Not implemented	Per-user forensic pixels would require a separate processing/caching pipeline.

Teaching-Platform Integration

Step	Owner	Action
1	Platform admin	Create an application with the DRM admin token; store the returned client secret server-side.
2	Instructor backend	Register media, upload through the returned PUT URL, then call the completion endpoint.
3	DRM worker	Validate, transcode, encrypt, package, and publish private output; emit signed status webhook.
4	Platform backend	Verify enrollment and sign a one-use assertion for the exact user, app, asset, and device.
5	Browser	Exchange assertion through the backend/proxy; configure Shaka with returned manifest and license URL.
6	Browser + DRM	Send bearer token on media/license requests, renew before expiry, heartbeat, and end on teardown.

Never embed an application client secret in a production frontend bundle. The demo Vite proxy injects credentials on the development server only.

Deployment Model

The local free stack uses PostgreSQL 16, Valkey 8, SeaweedFS 4.47, FFmpeg, Shaka Packager, Node.js, and Shaka Player. The production reference adds Caddy TLS, internal networks, required secrets, mandatory assertions, ClearKey disabled, and hardened application containers.

It is a single-host reference, not high availability. A real service still needs managed secrets, redundant data services, monitored backups, restore drills, outbound network policy, alerting, capacity planning, and incident response.

Verification Evidence

Check	Result	Evidence / limitation
TypeScript typecheck	Passed	Entire workspace, forced project build
Production build	Passed	API, worker, packages, and demo player
Unit tests	Passed	8 files, 26 tests
Compose parsing	Passed	Development definition validates
Live stack	Not verified this run	Docker Desktop did not become ready

Residual Risk and Production Gate

- Screen recording and external cameras cannot be fully prevented. Commercial DRM raises extraction cost; watermarking improves deterrence and attribution.
- A compromised browser/device can access decoded frames. Treat playback clients as partially hostile.
- Destination validation reduces webhook SSRF risk, but production should also enforce an outbound firewall or proxy allowlist.
- The current integration, media, and e2e files are configuration checks, not live full-stack proof.
- Single-host storage and database services are failure domains. Use redundancy and tested backups for paid content.

Required before high-value production use

- Obtain and integrate an approved Widevine/PlayReady/FairPlay license provider.
- Add live PostgreSQL, Valkey, S3, FFmpeg, Packager, browser, revocation, and recovery tests to CI.
- Move all production secrets and master keys to a managed secret/KMS system and exercise rotation.
- Deploy monitoring, immutable audit retention, alerting, backup/restore drills, and an incident-response runbook.
- Complete an independent threat-model review and penetration test.

Cost Statement

The repository and local open-source components can run without software license fees. Production is not honestly 100% free: hosting, bandwidth, storage, backups, monitoring, domain/TLS operations, engineering time, and commercial DRM agreements may cost money.

Bottom Line

The implementation now has a credible protected-delivery foundation: private encrypted media, short sessions, per-request authorization, atomic sharing controls, traceable overlay watermarking, safer uploads/webhooks, and a clear commercial-DRM boundary. It should be described as hardened development software until the live stack, real media pipeline, commercial license service, operations, and independent security review are complete.

Prepared from the repository state and verification results on 27 September 2026.